# TrustNote-AI: A Frozen-Backbone MobileNetV3 Classifier for Counterfeit Indian Banknote Screening on Commodity Hardware


Sai Shiva Ganesh
*Department of Computer Science and Engineering*
*SRM Institute of Science and Technology*
Chennai, India
sg3142@srmist.edu.in

Sricharan Suresh
*Department of Computer Science and Engineering*
*SRM Institute of Science and Technology*
Chennai, India
ss1833@srmist.edu.in



***Abstract*—Counterfeit banknote screening in low-resource settings runs on phone cameras and shared desktops, not on ultraviolet scanners. We present TrustNote-AI, a binary REAL/FAKE classifier for Indian banknotes built on MobileNetV3-Large with ImageNet-pretrained weights. The convolutional backbone stays frozen throughout training; only a 0.56 M-parameter classification head learns. On a held-out test set of 1121 images the model reaches 96.97% accuracy, 0.9555 $F_1$ on the counterfeit class, and 0.9959 ROC-AUC, while occupying 14.3 MB in 32-bit floating point and classifying an image in 9.1 ms on a desktop CPU. No GPU takes part at any stage: the reported run trained for nine epochs in 36.09 minutes on an Intel Core i5-1335U laptop CPU before early stopping, with the best validation $F_1$ of 0.9775 recorded at epoch four. The result carries a caveat that we treat as a central finding rather than a footnote: a frozen ImageNet feature extractor, which has never seen a security thread or an intaglio print, separates the two classes at 0.9959 AUC. Coarse photometric and print-process cues therefore drive the decision, not the security features a human examiner reads. We report the confusion matrix in full and specify the acquisition-controlled evaluation protocol that any deployment claim would require.**

***Index Terms*—counterfeit detection, Indian currency, transfer learning, MobileNetV3, edge inference, image classification, dataset bias**


## I. Introduction

The Reserve Bank of India reports counterfeit notes detected across the banking system every year, and the detected figure understates circulation because notes that never reach a bank counter never enter the count [1]. Verification hardware closes part of the gap. A ultraviolet lamp or a magnetic-ink reader costs more than a small merchant will spend and travels poorly, so the transactions most exposed to counterfeits are screened by eye or not at all.

A classifier that runs on a phone changes the economics of that screening. This paper reports what such a classifier achieves, and what its headline accuracy conceals.

We train MobileNetV3-Large [2] on a denomination-stratified corpus of real and counterfeit Indian banknote photographs and evaluate it on a held-out 15% partition. Three design decisions define the system. The backbone remains frozen, so training touches 16% of the parameters. A weighted sampler corrects a 2:1 class imbalance during training while the test partition retains its natural prior. The augmentation pipeline models photographic nuisance variables, including perspective, blur, and occlusion, rather than generic geometric jitter.

The contributions of this paper are:

1) A complete, reproducible pipeline for binary Indian banknote authentication that trains in under ten epochs and deploys in 14.3 MB, with audit, split, training, evaluation, and inference stages released as code.
2) A measured result on 1121 held-out images: 96.97% accuracy and 0.9959 ROC-AUC, reported with the full confusion matrix and per-class breakdown rather than accuracy alone.
3) An analysis showing that a frozen ImageNet backbone suffices for this performance, and an argument that this fact bounds what the reported numbers can be claimed to demonstrate about counterfeit detection as opposed to image-source discrimination.
4) An evaluation protocol, specified in Section VI-B, that would separate the two.

## II. Related Work

Automated banknote authentication divides into three families.

**Sensor-based methods** read physical properties: ultraviolet fluorescence, infrared absorption, magnetic ink response, and paper thickness. These methods target the substrate that counterfeiters find hardest to reproduce, and they achieve the accuracy that central banks require. They also require dedicated hardware, which rules them out for the deployment target considered here.

**Hand-engineered visual features** extract security elements from photographs: the security thread, the latent image, microlettering, the watermark region, and the intaglio texture

TABLE I
PARTITION SIZES.

| Partition | `REAL` | `FAKE` | Total |
|---|---|---|---|
| Training | 3,472 | 1,759 | 5,231 |
| Validation | 744 | 377 | 1,121 |
| Testing | 744 | 377 | 1,121 |
| **Total** | **4,960** | **2,513** | **7,473** |

of the portrait. Systems in this family localize a region of interest, apply a filter bank or texture descriptor, and classify the response. Performance depends on localization accuracy, which degrades under the perspective and lighting variation typical of handheld capture [3].

**Learned representations** replace the feature engineering with a convolutional network. Transfer learning from ImageNet [4] dominates this family because banknote corpora are small by computer vision standards. Yosinski et al. [5] established that early convolutional layers transfer across tasks with little loss, which motivates freezing them when target data is scarce. MobileNetV2 and V3 [2], [6] extend the approach to mobile inference budgets through inverted residual blocks and squeeze-and-excitation attention [7].

Work in the third family reports accuracies above 95% with regularity. Torralba and Efros [8] give the reason to read those numbers with care: when the two classes in a corpus were photographed under different conditions, a classifier separates the conditions and the reported accuracy measures nothing about the intended task. Section VI applies that argument to our own result.

## III. DATASET

### A. Composition

The corpus contains photographs of Indian banknotes in seven denominations (Rs. 10, Rs. 20, Rs. 50, Rs. 100, Rs. 200, Rs. 500, and Rs. 2000), organized as a two-level hierarchy of class and denomination. The prediction task collapses the denomination axis: the label space is $\mathcal{Y} = \{\texttt{REAL}, \texttt{FAKE}\}$, encoded as $\texttt{REAL} \mapsto 0$ and $\texttt{FAKE} \mapsto 1$, with `FAKE` treated as the positive class for precision and recall.

The corpus holds 7473 images, 4960 labeled `REAL` and 2513 labeled `FAKE`. A stratified 70/15/15 split produces the partitions in Table I. The `REAL`-to-`FAKE` ratio holds at 1.97:1 across all three.

The test partition stayed out of both training and model selection. Every number in Section V comes from a single pass over it after the checkpoint was fixed.

### B. Audit

Every image passes an audit stage before splitting. The audit walks the directory tree, computes a SHA-256 digest per file, opens each image with a verification pass, and records width, height, color mode, and container format. The stage emits three artifacts: a per-image inventory, a list of files that failed to decode, and a list of digest collisions.

TABLE II
TRAINING AUGMENTATION PIPELINE. VALIDATION AND TEST IMAGES RECEIVE RESIZE AND NORMALIZATION ONLY.

| Operation | Parameters | Models |
|---|---|---|
| Resize | $256 \times 256$ | — |
| RandomResizedCrop | scale $[0.85, 1.0]$ | camera distance |
| RandomRotation | $\pm 5°$ | handheld tilt |
| RandomAffine | shift 3%, shear 2° | framing drift |
| RandomPerspective | distortion 0.15, $p$=0.30 | off-axis capture |
| ColorJitter | brightness/contrast 0.15 | lighting |
| GaussianBlur | $k$=3, $p$=0.20 | motion blur |
| RandomErasing | 2–8% area, $p$=0.15 | finger, shadow |

Exact-duplicate detection at the file level catches copied files. It does not catch two photographs of the same physical note taken seconds apart, which produce different digests and near-identical content. Section VII returns to this.

### C. Partitioning

The split runs per class and per denomination with a fixed seed of 42. Within each of the fourteen leaf directories, a 70% training partition is drawn first, and the 30% remainder is halved into validation and test. Stratifying at the denomination level keeps the denomination distribution stable across the three partitions, which prevents the classifier from being evaluated on a denomination mix it never trained on.

The split unit is the image file. It is not the physical note, and it is not the capture session.

## IV. METHOD

### A. Preprocessing

All images convert to RGB, resize to $224 \times 224$, and normalize with ImageNet channel statistics ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$). Matching the normalization used to pretrain the backbone keeps the frozen features in the input distribution they were fitted on.

Resizing to a fixed square discards the aspect ratio. Indian denominations differ in physical dimensions, so this step removes a cue that would otherwise identify the denomination. The binary REAL/FAKE task does not need that cue, and withholding it stops the head from routing its decision through denomination.

### B. Augmentation

Training images pass through a pipeline that models handheld capture rather than generic geometric perturbation. Table II lists the operations and the nuisance variable each one targets.

The parameter ranges stay narrow. Rotation caps at five degrees and color jitter at 15%, because a banknote photographed for verification is held roughly square-on and roughly in frame. Wider ranges would inject variation the deployment distribution does not contain. Random erasing [9] applies after tensor conversion and before normalization, so the erased region carries raw-scale noise.

TABLE III
REPLACEMENT CLASSIFICATION HEAD. ALL PARAMETERS LISTED ARE TRAINABLE; THE BACKBONE IS FROZEN.

| Layer | Shape | Parameters |
|---|---|---|
| Linear | $960 \to 512$ | 492,032 |
| BatchNorm1d | 512 | 1,024 |
| Hardswish | — | 0 |
| Dropout | $p = 0.30$ | 0 |
| Linear | $512 \to 128$ | 65,664 |
| BatchNorm1d | 128 | 256 |
| Hardswish | — | 0 |
| Dropout | $p = 0.20$ | 0 |
| Linear | $128 \to 2$ | 258 |
| **Trainable total** | | **559,234** |
| Frozen backbone | | 2,971,952 |

### C. Architecture

The backbone is MobileNetV3-Large [2] with the default ImageNet weights. The original classifier is discarded and replaced with the head in Table III, which takes the 960-dimensional pooled feature vector and emits two logits.

Hardswish activations match the backbone. Batch normalization before each nonlinearity stabilizes the head while the features feeding it stay fixed. Dropout tapers from 0.30 to 0.20 across the two hidden layers.

The trainable fraction is 559,234 of 3,531,186 parameters, or 15.84%. We fitted no convolutional filter on currency data. All of them arrived from ImageNet and stayed there.

### D. Class Imbalance

The training loader draws through a `WeightedRandomSampler` with weights inversely proportional to class frequency, sampling with replacement for one epoch-equivalent of draws. Each epoch therefore presents a balanced stream, and the majority `REAL` class stops dominating the gradient.

Validation and test loaders sample sequentially without weighting. The reported metrics reflect the natural 2:1 prior rather than a balanced one.

### E. Optimization

Training minimizes cross-entropy with AdamW [10] at an initial learning rate of $10^{-4}$ and weight decay of $10^{-4}$, decayed by a cosine schedule [11] toward $\eta_{\min} = 10^{-6}$ over a 30-epoch horizon. Batch size is 32. Gradients clip to unit $\ell_2$ norm. The trainer wraps the forward pass in an automatic mixed precision scope [12] with a gradient scaler that unscales before the clip, so the threshold applies to true gradient magnitudes whenever that path is active.

Model selection tracks validation $F_1$ on the `FAKE` class rather than accuracy, which would be inflated by the majority class. Training halts when validation $F_1$ fails to improve for five consecutive epochs, and the checkpoint with the best validation $F_1$ is restored for testing. We fix random seeds at 42 for PyTorch and CUDA.

TABLE IV
TRAINING HISTORY. THE SELECTED CHECKPOINT IS EPOCH 4.

| Epoch | Train loss | Val loss | Val acc. | Val $F_1$ |
|---|---|---|---|---|
| 1 | 0.2528 | 0.1368 | 0.9436 | 0.9212 |
| 2 | 0.1506 | 0.0900 | 0.9687 | 0.9550 |
| 3 | 0.1062 | 0.0757 | 0.9758 | 0.9649 |
| **4** | **0.0864** | **0.0530** | **0.9848** | **0.9775** |
| 5 | 0.0947 | 0.0529 | 0.9821 | 0.9738 |
| 6 | 0.0758 | 0.0466 | 0.9839 | 0.9763 |
| 7 | 0.0770 | 0.0533 | 0.9830 | 0.9750 |
| 8 | 0.0638 | 0.0464 | 0.9794 | 0.9700 |
| 9 | 0.0598 | 0.0453 | 0.9830 | 0.9749 |

TABLE V
TEST PERFORMANCE ON 1121 HELD-OUT IMAGES. PRECISION, RECALL, AND $F_1$ ARE COMPUTED WITH `FAKE` AS THE POSITIVE CLASS.

| Metric | Value |
|---|---|
| Accuracy | 0.9697 |
| Precision | 0.9432 |
| Recall | 0.9682 |
| $F_1$ | 0.9555 |
| ROC-AUC | 0.9959 |

The pipeline gates mixed precision on `torch.cuda.is_available()` and otherwise falls back to full-precision CPU execution. The reported run took the CPU path on an Intel Core i5-1335U and finished nine epochs in 36.09 minutes, so the mixed-precision branch stayed inactive and the scaler ran as a no-op. Inference carries no CUDA dependency either, and the latency figures in Section V-C come from a CPU-only build.

Training the full pipeline in 36 minutes on a mid-range laptop CPU follows from the frozen backbone. Gradients flow through 559,234 parameters rather than 3.5 M, which puts reproduction of this work within reach of any machine that can run the dataset through a forward pass.

## V. RESULTS

### A. Convergence

Training stopped at epoch nine of a 30-epoch budget. Table IV gives the trajectory. The best validation $F_1$ of 0.9775 arrived at epoch four; the five epochs that followed produced no improvement, and early stopping triggered.

Two features of the trajectory deserve comment. Validation $F_1$ passes 0.92 in a single epoch, before the head has seen the training set twice. And after epoch four, training loss keeps falling while validation $F_1$ plateaus, the standard signature of a head beginning to fit noise that the frozen features cannot resolve.

### B. Test Performance

Table V reports threshold-dependent metrics at the default argmax decision rule, equivalent to a 0.5 probability threshold.

TABLE VI
CONFUSION MATRIX, 1121 TEST IMAGES.

| | | Predicted | |
|---|---|---|---|
| | | **REAL** | **FAKE** |
| **Actual** | **REAL** | 722 | 22 |
| | **FAKE** | 12 | 365 |

TABLE VII
SINGLE-IMAGE INFERENCE COST, CPU ONLY, BATCH SIZE 1, 224 × 224 INPUT, PYTORCH 2.10 ON AN AMD RYZEN 9 7900X WITH 12 THREADS.

| **Quantity** | **Value** |
|---|---|
| Parameters (total) | 3,531,186 |
| Parameters (head) | 559,234 |
| Checkpoint size | 14.34 MB |
| Latency, median | 9.1 ms |
| Latency, 95th pct. | 17.6 ms |
| Latency, min / max | 7.9 / 64.6 ms |

The error budget is 34 images out of 1121. Twenty-two genuine notes were flagged as counterfeit, and twelve counterfeits passed as genuine.

The asymmetry matters for deployment. A false positive sends a genuine note to manual inspection, which costs a merchant time. A false negative accepts a counterfeit, which costs the merchant the face value of the note. The current threshold produces almost twice as many false positives as false negatives, and the ROC-AUC of 0.9959 indicates headroom to move the operating point further in that direction. Section VI-B treats threshold selection as an open item rather than a solved one.

### C. Deployment Cost

The trained network holds 3,531,186 parameters, of which the replacement head contributes 559,234 (15.84%). The serialized checkpoint occupies 14.34 MB on disk, against 14.12 MB of raw 32-bit parameter data.

Table VII reports single-image CPU latency measured on the released checkpoint. We loaded the weights with strict state-dict matching, warmed up for ten forward passes, and timed fifty passes at batch size one.

These figures come from a desktop part, while the training run used a laptop i5-1335U. A 9.1 ms median leaves room for the gap: even a tenfold penalty on a phone SoC keeps a single verification under 100 ms. INT8 post-training quantization [13] would cut the footprint to roughly 3.6 MB, and we have measured neither its latency nor its effect on accuracy.

## VI. DISCUSSION

### A. What the Frozen Backbone Implies

The central result of this paper is that 96.97% accuracy arrived without updating a single convolutional filter.

MobileNetV3's ImageNet features encode edges, textures, color distributions, and object-level shape statistics. Nothing in the pretraining corpus contains a security thread, an intaglio ridge, microlettering, or a watermark. A trained human examiner separates genuine notes from counterfeits on those four cues, and they demand the fine-grained, spatially localized evidence that a frozen generic feature extractor is least equipped to supply.

A linear-plus-MLP probe on those frozen features reaches 0.9959 ROC-AUC. The straightforward reading is that the two classes in this corpus are separable by coarse, global, low-level statistics. Candidate sources for such a signal include print process (offset or inkjet reproduction versus intaglio), paper reflectance, color gamut differences from the reproduction pipeline, and the capture conditions under which each class was photographed.

Some of those are legitimate counterfeit evidence. Print-process artifacts are real forensic signal, and a model keying on them is doing useful work. Capture conditions are not. Should the two classes have been photographed on different equipment or under different room lighting, the classifier reaches these numbers by identifying the photograph rather than the note [8], and the reported metrics transfer to no deployment at all.

Our evidence does not separate the two cases, so we describe the system as a class separator for this corpus and stop short of calling it a counterfeit detector.

### B. Diagnostic Experiments

Four experiments would resolve the question, ordered by cost.

**Grad-CAM attribution.** Compute class activation maps [14] over the test partition and measure what fraction of the attribution mass falls on security-feature regions (thread, watermark window, portrait intaglio) against background and margin. Attribution concentrated on borders or uniform paper indicates capture-artifact reliance. This costs an afternoon and should run first.

**Backbone fine-tuning.** Unfreeze the backbone and fine-tune at a reduced learning rate. The code already exposes `unfreeze_backbone()`, and the comment in `train.py` labels the current run "Stage 1," so the two-stage schedule was designed and left unexecuted. If fine-tuning yields little gain, the frozen features already saturate the available signal, which supports the coarse-cue hypothesis.

**Cross-source evaluation.** Partition by capture session or device rather than by file, train on one source, and test on a held-out source. A drop from 0.97 accuracy toward chance under this protocol identifies source discrimination. This is the decisive experiment, and it requires acquisition metadata the current corpus does not carry.

**Note-level splitting.** Group all photographs of each physical note into a single partition. File-level splitting places two shots of one note into training and test, which inflates the reported metrics by an amount we cannot currently bound.

### C. Threshold Selection

The system decides at the argmax, which is the correct rule only when the two error types carry equal cost and the softmax outputs are calibrated. Neither holds. A false negative costs the note's face value; a false positive costs a few seconds. Modern networks also report overconfident softmax scores [15], so a merchant should not read the 0.99 confidence in our example output as a 1-in-100 error rate without temperature scaling fitted on the validation partition.

A deployed version should fit a temperature on validation data, then select the threshold that minimizes expected cost under a stated cost ratio, and report performance at that operating point rather than at 0.5.

## VII. Limitations

- **Provenance.** The corpus lacks acquisition metadata, so we cannot verify that genuine and counterfeit images share capture conditions. This is the binding limitation on every number in Section V.
- **Splitting granularity.** Partitioning by file rather than by physical note permits near-duplicate leakage between training and test. The audit stage detects byte-identical files only.
- **Counterfeit quality.** The corpus does not stratify counterfeits by sophistication. Performance against high-quality reproductions, which is the case that matters, is unmeasured.
- **Denomination coverage.** Results aggregate across seven denominations, and we do not break performance out by denomination. The corpus also retains Rs. 2000 images, a denomination the Reserve Bank has since withdrawn from circulation, which dilutes the benchmark's relevance by an unmeasured amount.
- **Single architecture.** We ran one configuration and report it. No ablation isolates the contribution of the custom head, the weighted sampler, or the individual augmentations.
- **Latency measured off-target.** We timed inference on a desktop Ryzen 9 7900X, not on the i5-1335U that trained the model and not on a phone SoC, which is the deployment target the design assumes.

## VIII. Conclusion

TrustNote-AI classifies Indian banknote photographs as genuine or counterfeit at 96.97% accuracy and 0.9959 ROC-AUC on a 1121-image held-out set, in a 14.3 MB model that trains in 36 minutes on a laptop CPU and answers in 9.1 ms. The engineering result is solid, and the pipeline is reproducible end to end on hardware a student already owns.

The scientific result is more interesting than the accuracy figure and points in a less comfortable direction. Those numbers came from a network whose every convolutional filter was fitted on ImageNet photographs of animals and household objects, with only a half-million-parameter head trained on currency. Security features are not in that representation. Whatever separates these two classes at 0.9959 AUC is coarse enough for generic ImageNet features to capture it, which places print-process artifacts and capture conditions ahead of intaglio texture as the likely explanation.

Separating those two possibilities requires the cross-source evaluation described in Section VI-B, and until that experiment runs, the honest claim is narrower than the headline. The model separates the classes in this corpus. Whether it detects counterfeits remains open.

## References


[1] Reserve Bank of India, "Annual report: Counterfeit notes detected in the banking system," https://rbi.org.in, 2024, [CITE: replace with the specific RBI Annual Report year and table used].

[2] A. Howard, M. Sandler, G. Chu, L.-C. Chen, B. Chen, M. Tan, W. Wang, Y. Zhu, R. Pang, V. Vasudevan, Q. V. Le, and H. Adam, "Searching for MobileNetV3," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019, pp. 1314–1324.

[3] C. author], "[cite: a survey or prior work on indian banknote authentication using image processing]," *[CITE: journal]*, [CITE: year], placeholder: insert domain-specific prior work before submission.

[4] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009, pp. 248–255.

[5] J. Yosinski, J. Clune, Y. Bengio, and H. Lipson, "How transferable are features in deep neural networks?" in *Advances in Neural Information Processing Systems (NeurIPS)*, 2014, pp. 3320–3328.

[6] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "MobileNetV2: Inverted residuals and linear bottlenecks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510–4520.

[7] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 7132–7141.

[8] A. Torralba and A. A. Efros, "Unbiased look at dataset bias," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2011, pp. 1521–1528.

[9] Z. Zhong, L. Zheng, G. Kang, S. Li, and Y. Yang, "Random erasing data augmentation," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2020, pp. 13 001–13 008.

[10] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *International Conference on Learning Representations (ICLR)*, 2019.

[11] ——, "SGDR: Stochastic gradient descent with warm restarts," in *International Conference on Learning Representations (ICLR)*, 2017.

[12] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu, "Mixed precision training," in *International Conference on Learning Representations (ICLR)*, 2018.

[13] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.

[14] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual explanations from deep networks via gradient-based localization," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2017, pp. 618–626.

[15] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proceedings of the 34th International Conference on Machine Learning (ICML)*, 2017, pp. 1321–1330.